

Solitaire Developer Case Study — AI Collaboration Log

This document summarizes the development discussion and architectural decisions made during implementation of the Solitaire Developer Case Study prototype. The conversation focused on architecture validation, reusable system design, undo-flow correctness, and project scoping.

1. Initial Scope Definition

- Avoid implementing full Solitaire gameplay logic.
- Focus on card movement between stacks and undo functionality.
- Use Unity 2D setup with placeholder visuals.
- Prioritize clean architecture and reusable systems.

2. Undo System Design

- Compared two approaches: single last-move storage vs move history stack.
- Selected extensible move-history approach using TurnInfo records.
- Undo implemented through reversing source/destination stacks.
- Architecture kept scalable while remaining small for assignment scope.

3. Drag-and-Drop System

- Implemented isolated reusable drag-and-drop package.
- Used Unity EventSystem + Physics2D raycasting.
- Separated gameplay logic from drag handlers.
- System exposes event-driven API using drag events and drag result data.

4. Architecture Refinement

- Initial Bootstrap approach was simplified.
- Introduced GameCompositionRoot to compose dependencies.
- Separated responsibilities into:
 - - Bootstrap layer
 - - Composition layer
 - - Gameplay controller
 - - View layer
 - - Drag-and-drop package

5. Code Review Feedback

- Removed MonoBehaviour misuse from DragAndDropService.
- Improved event ownership and disposal handling.
- Separated gameplay concerns from reusable package concerns.
- Reviewed SOLID alignment and dependency boundaries.

6. Undo Edge Case Investigation

- Tested scenario involving move → undo → move → undo.
- Discovered draggable interaction state desynchronization.
- Fixed by synchronizing draggable dock state after undo.
- Validated behavior using runtime logs.

7. README and Documentation

- Created architecture-focused README.
- Documented AI-assisted areas transparently.
- Added technical reasoning and future improvements.
- Documented tested edge cases and implementation scope decisions.

The final prototype intentionally focuses on clean architecture, modularity, and realistic production-oriented structure rather than feature quantity. AI assistance was used primarily for architecture validation, code review, edge-case analysis, and documentation support.

Generated: 2026-05-07 00:15